CSE 5525: Assignment 3

Qifan Wen

Resource	VM 1 (Parts 1 & 2)	VM 2 (Part 3)
OS	Ubuntu 22.04 LTS (Linux 5.4, x86_64)	Ubuntu 24.04 LTS (Linux 6.8, x86_64)
GPU	NVIDIA A40 (46 GB VRAM)	NVIDIA RTX 5090 (32 GB VRAM)
GPU driver	535.129.03	590.48.01
CPU	AMD EPYC 7H12 64-Core (128 threads)	AMD EPYC 9654 96-Core (384 threads)
RAM	1 TB	755 GB
Storage	50 GB overlay (NVMe)	60 GB overlay (NVMe)
Software	Python 3.10, PyTorch 2.10, Transformers 5.2	Python 3.12, PyTorch 2.10, Transformers 5.3
CUDA / cuDNN	12.2 / 9.1	12.8 / 9.1
Batch sizing	Automatic per-config tuning to maximize VRAM utilization	

Table 1: Hardware and software environment used for all experiments. Parts 1 & 2 (T5 training) ran on VM 1; Part 3 (LLM prompting) ran on VM 2.

1 Data Statistics and Processing (8pt)

Table 2 reports the raw dataset statistics before any model-specific preprocessing. All token counts use the T5 tokenizer (`T5TokenizerFast` from `google-t5/t5-small`). Vocabulary sizes are computed over unique whitespace-delimited words.

Statistics Name	Train	Dev
Number of examples	4225	466
Mean sentence length	18.1	18.1
Mean SQL query length	217.4	211.1
Vocabulary size (natural language)	868	444
Vocabulary size (SQL)	644	393

Table 2: Data statistics before any pre-processing. Token counts are measured using the T5 tokenizer.

Table 3 shows per-model statistics after preprocessing. The T5 fine-tuned model prepends table names from the flight database schema to each natural language input (e.g., "tables: city, airport, flight, ..."), adding approximately 93 tokens per example. The T5 from-scratch model receives raw NL input without any schema augmentation. Both models restrict the decoder output vocabulary to 598 SQL-relevant tokens (SQL keywords, table/column names, operators, and literals extracted from the training data).

Statistics Name	Train	Dev
T5 Fine-Tune		
Mean encoder input length	111.1	111.1
Mean decoder target length	217.4	211.1
Encoder token vocab size	815	493
Decoder token vocab size	556 (598 restricted)	396 (598 restricted)
T5 From Scratch		
Mean encoder input length	18.1	18.1
Mean decoder target length	217.4	211.1
Encoder token vocab size	792	466
Decoder token vocab size	556 (598 restricted)	396 (598 restricted)

Table 3: Data statistics after pre-processing.

2 T5 Fine-tuning and Training From Scratch (8pt)

2.1 Supervised Fine-tuning (SFT)

Table 4 describes the data processing, tokenization, architecture, and hyperparameters for the fine-tuned and from-scratch T5 models.

Design choice	Description
Data processing	**Fine-tuned:** schema table names prepended to NL input. **From scratch:** raw NL input, no schema augmentation.
Tokenization	`T5TokenizerFast` from `google-t5/t5-small`; restricted decoder vocabulary (598 SQL tokens). Shared across both settings.
Architecture	**Fine-tuned:** T5-small (60M params) with restricted vocab head; also explored LoRA and MLP heads. **From scratch:** same T5-small architecture from random initialization.
Varying parameters	**Fine-tuned:** LR, batch size, dropout, schema, encoder freezing, adaptation method, max epochs, warmup, eval frequency, patience, wall-clock budget. **From scratch:** LR, batch size, dropout, label smoothing, max epochs, warmup, eval frequency, patience, wall-clock budget.
Constant parameters	Optimizer, scheduler, gradient clipping, loss function, beam search decoding, max generation length, seed, restricted decoder vocabulary.

Table 4: Design choices for T5 fine-tuned and from-scratch models.

2.2 Direct Preference Optimization (DPO)

Table 5 describes the DPO post-training pipeline applied on top of the best SFT model: the T5-base "Base sweep best" checkpoint (`google-t5/t5-base`, 222M params; see Table 12, F1 = 85.96%, SQL EM = 73.39%). DPO *(Rafailov et al., 2023)* optimizes the policy to prefer "chosen" over "rejected" completions without training a separate reward model, using the SFT checkpoint as both initialization and frozen reference. Results are reported in Section 4.

Design choice	Description
Base model	T5-base "Base sweep best" (222M params, full fine-tune, restricted vocab, schema augmentation, LR = 6.19e-5, label smoothing = 0.15). Used as both policy initialization and frozen reference.
Preference data	9,801 triplets (NL, chosen SQL, rejected SQL). **Phase A:** model-generated negatives — up to 10 candidates per query via diverse beam search, classified by execution correctness against `flight_database.db`. **Phase B:** execution-aware perturbations of gold SQL (city swaps, date changes, operator flips), verified to produce different record sets. In-memory SQLite for fast evaluation (46K+ queries in ∼20 min).
Loss	$\mathcal{L}_{\text{DPO}} = -\log\sigma(\beta\,[\log\pi_\theta(y_w\|x) - \log\pi_{\text{ref}}(y_w\|x) - \log\pi_\theta(y_l\|x) + \log\pi_{\text{ref}}(y_l\|x)])$. Log probs computed via restricted SQL vocab (598 tokens); OOV tokens masked to zero.
Varying parameters	$\beta \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$, LR $\in [10^{-7}, 5\times10^{-5}]$, weight decay, gradient clipping, architecture (full fine-tune vs. LoRA r=16 on q/v or q/k/v/o). W&B random sweep.
Constant parameters	Optimizer (AdamW), beam search decoding (num_beams = 4), restricted decoder vocabulary, schema augmentation, eval every epoch, patience = 5 eval cycles.

Table 5: Design choices for DPO post-training on T5-base.

2.3 RL Fine-Tuning (PPO, GRPO, CISPO)

We explored three online reinforcement learning algorithms applied to the best T5-base SFT checkpoint (Table 12, F1 = 85.96%) via LoRA adapters ($r = 32$, $\alpha = 64$, targeting q/k/v/o projections, 3.08% trainable parameters). This investigation was motivated by recent work on RL-based LLM alignment: `GRPO` from DeepSeekMath, `CISPO` from MiniMax-M1, and classic `PPO` with a learned value head baseline. To our knowledge, no published work applies these algorithms to T5 encoder-decoder models for NL-to-SQL.

Methodology. For each query, $G = 4$ SQL completions are generated via constrained decoding (temperature = 0.7, top-k = 50). Each completion receives a graded execution reward: +1.0 (exact record match), +0.5 (partial overlap), −0.5 (no overlap), −1.0 (SQL error). Advantages are computed group-relative for GRPO/CISPO and via a learned value head for PPO. Training uses a random subset of 200–500 queries per epoch with early stopping on dev Record F1.

- **PPO**: Clipped surrogate objective with a learned value head (MLP on mean-pooled encoder hidden states) as baseline; entropy bonus ($\alpha = 0.01$) for exploration.
- **GRPO**: PPO-style clipped surrogate with group-relative advantage normalization (no critic network). KL penalty ($\beta = 0.05$).
- **CISPO**: Detached clamped importance sampling weights that preserve gradient flow for all tokens. No KL penalty ($\beta = 0$).

Results. Table 6 reports dev set metrics for all three algorithms.

Analysis. None of the three RL algorithms improved over the SFT baseline. We identify four contributing factors:

System	SQL EM (%)	Record EM (%)	Record F1 (%)	KL Drift	Time
SFT baseline (T5-base)	73.39	84.76	85.96	–	–
PPO + Value Head	73.39	84.76	85.96	-6.55	46 min
GRPO	73.39	84.76	85.96	-9.88	35 min
CISPO	73.39	84.76	85.96	+0.21	45 min

Table 6: RL fine-tuning results on T5-base (Section 2.3). All three algorithms match but do not improve the SFT baseline (85.96% F1). KL Drift = KL divergence from the reference policy at early stopping.

1. **Reward sparsity.** The base model already achieves F1 = 85.96%. Most generated completions receive +1.0 reward, creating 50–64% "dead groups" (zero reward variance). The RL gradient signal is too sparse to improve an already-strong model.
2. **Single-step ratio reduction.** With one gradient step per rollout batch, the importance sampling ratio is always 1.0. PPO clipping (clip_fraction = 0.000 across all runs) never activates. All three algorithms effectively reduce to vanilla REINFORCE with different advantage baselines.
3. **Policy drift without improvement.** PPO and GRPO show increasing negative KL divergence (-6.55 and -9.88 by early stopping), meaning the policy explores lower-probability regions without finding better outputs. CISPO's detached clipping keeps KL near zero (+0.21) but makes effectively zero policy updates (loss ≈ -0.008).
4. **Coarse reward granularity.** Four discrete reward levels cannot distinguish "almost correct" from "completely wrong" SQL. A continuous reward based on record set Jaccard similarity might provide finer gradient signal.

These results suggest that online RL is not effective for improving already-strong NL-to-SQL models on small datasets ($\sim$4K queries). The restricted 598-token SQL vocabulary already constrains the output space, limiting the model's ability to explore meaningfully different SQL structures. Rejection sampling fine-tuning (generate, filter by reward, SFT on filtered outputs) may be more practical for post-training improvement in this setting.

3 Large Language Model (LLM) Prompting (14pt)

3.1 In-Context Learning (ICL)

Instructions: Provide in Table 7 the instruction prompt(s) that you used for ICL.

If the prompt you used for zero- and few-shot prompting is identical, except for the examples, there's no need to repeat it. If you made small modifications between zero- and few-shot, please provide them separately. For all entries, you need to specify the corresponding values of k.

Shot	Prompt

$k = 0$	`<start_of_turn>user`
	Database schema:
	CREATE TABLE city (...); ... (25 tables)
	
	Translate the following question into SQL. Output only the SQL query.
	
	Question: {query}
	SQL:`<end_of_turn>`
	`<start_of_turn>model`

$k \geq 1$	`<start_of_turn>user`
	Database schema:
	CREATE TABLE city (...); ... (25 tables)
	
	Translate the following question into SQL. Output only the SQL query.
	
	Question: {example$_1$ NL}
	SQL: {example$_1$ SQL}
	
	... (repeated k times)
	
	Question: {query}
	SQL:`<end_of_turn>`
	`<start_of_turn>model`

Table 7: Instruction prompts used for zero- and few-shot prompting. The prompt uses the Gemma chat format. The database schema provides all 25 table definitions as CREATE TABLE DDL. For $k \geq 1$, examples are inserted between the instructions and the target question. The same template is used for both random and BM25 example selection; only the choice of examples differs.

Example selections: Please provide a clear, detailed, and succinct description of how you selected the examples when $k > 0$.

We evaluated two example selection strategies for $k > 0$:

Random selection: k examples are sampled uniformly at random from the 4,225 training pairs. The same fixed set is used for all queries in a given run (seed = 42).

BM25 per-query selection: We build a BM25Okapi index (`rank_bm25`) over whitespace-tokenized, lowercased training questions. For each dev/test query, the index scores all training questions by term overlap and we select the top-k highest-scoring examples. This provides each query with semantically similar demonstrations whose SQL patterns are more likely to transfer. BM25 selection at $k = 3$ improved Record F1 from 11.96% (random) to 17.35% (+45% relative).

3.2 Best Prompt and Ablation Study

Instructions: Report the best prompt you used in Table 8. If the best prompt you used is the same as the one specified in Table 7, you can just copy the best prompt and label it. If it is different (e.g. you designed another prompt that is better), you should clearly describe how you created it and what are the methods you used in the caption.

You will also need to clearly and succintly describe in Table 9 the ablation experiments that you performed by removing different parts of the prompt. For that, you need to first highlight the parts of the prompt that you ablated for each experiment in a distinct color[1], as shows the placeholder example in Table 7, and second, provide the description by referring to the highlighted part. When reporting your results in Table 14, you will need to refer to your ablations variants.

Prompt

`<start_of_turn>user`
Database schema:
CREATE TABLE city (...); ... (25 tables)
 	
Translate the following question into SQL. Output only the SQL query.
 	
Question: {BM25-selected example$_1$ NL}
SQL: {BM25-selected example$_1$ SQL}
 	
Question: {BM25-selected example$_2$ NL}
SQL: {BM25-selected example$_2$ SQL}
 	
Question: {BM25-selected example$_3$ NL}
SQL: {BM25-selected example$_3$ SQL}
 	
Question: {query}
SQL:`<end_of_turn>`
`<start_of_turn>model`

Table 8: The best prompt (BM25 $k = 3$). Same template as Table 7 with $k = 3$ BM25 per-query selected examples. Blue = database schema (ablated in "No schema"). Green = task instructions (ablated in "No instructions").

[1] https://www.overleaf.com/learn/latex/Using_colors_in_LaTeX

Color	Description
ForestGreen	**No instructions:** Removed the task instruction line ("Translate the following question into SQL. Output only the SQL query."). Without this instruction, the model has no explicit directive to produce SQL, resulting in 100% SQL errors and F1 dropping from 17.35% to 11.80%.
Blue	**No schema:** Removed the full database schema (all 25 CREATE TABLE DDL statements). Without schema context, the model cannot reference correct table and column names, increasing the SQL error rate from 86% to 99% and reducing F1 from 17.35% to 11.80%.
–	**Schema only:** Removed both instructions and all examples ($k = 0$), keeping only the schema. The model receives table definitions but no task framing or demonstrations. Despite the lowest error rate (70%), F1 is worst at 11.37% because the model generates syntactically valid but semantically incorrect SQL.

Table 9: Ablation variants applied to the best prompt (BM25 $k = 3$). Each condition removes one or more components to measure its contribution to overall performance.

4 Results and Analysis (20pt)

Quantitative Results: Use Table 11, Table 12, and Table 14 to report your test and development set results. Your test results should match with the results on gradescope. For the development set, you should also report results from experiments you conducted to arrive at your final configuration. When reporting experiments, you should replace "variant" with brief and meaningful descriptions of whatever hyperparameter or setting that you varied. For ICL, you should specify what is the parameter k used, and what the full model corresponds to. For T5, the full model refers to the best model you described in Section 2. For T5, if you experimented with different design choices, you can add rows specifying the variants and what you tried. The text in gray is only for example purpose, and should be removed and replaced with your own choices. You may add more rows if needed.

Parameter	Value
Model	T5-small (`google-t5/t5-small`, 60M params)
Decoder vocabulary	Restricted (598 SQL tokens)
Optimizer	AdamW (weight decay = 0.01)
Scheduler	Cosine with warmup
Gradient clipping	Max norm = 1.0
Loss	Cross-entropy
Decoding	Beam search (num_beams = 4)
Max generation length	512 tokens
Seed	42
Test batch size	16
SQL evaluation threads	128

Table 10: Constant hyperparameters shared across all T5 training runs (Parts 1 & 2).

Query EM correction. The T5 SentencePiece tokenizer collapses `" , "` (space-comma-space) to `", "` (comma-space) during `batch_decode`, causing nearly every predicted query to fail string equality regardless of semantic correctness. Experimental variant prediction files were not retained, so their corrected Query EM cannot be recovered (marked "–"). For the final best fine-tuned and from-scratch models, a regex post-processing step restores the original comma spacing, yielding corrected Query EM of **58.80%** and **50.43%** respectively. Record F1 and Record EM are unaffected since the spacing difference does not change SQL execution results.

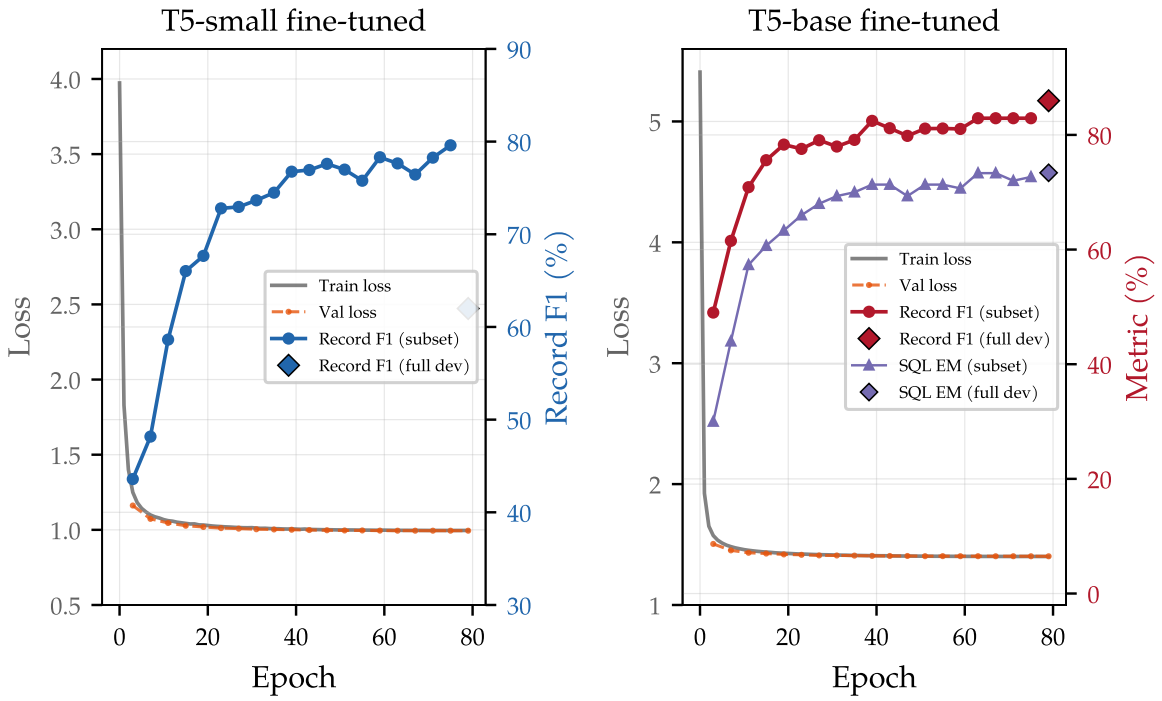

Figure 1: Training curves for two best fine-tuned models. **Left:** T5-small "Restricted v3" (full fine-tune, LR = 3e-4, batch = 64, schema augmentation, dropout = 0.1, 79 epochs; see Table 11). **Right:** T5-base "Base sweep best" (full fine-tune, LR = 6.19e-5, batch = 32, schema augmentation, dropout = 0.05, label smoothing = 0.15, 79 epochs; see Table 12). Gray: training loss (left axis). Orange dashes: validation loss (left axis). Blue/red circles: Record F1 on a dev subset (right axis). Purple triangles (T5-base only): SQL Exact Match on a dev subset (right axis). Diamonds: final full dev set metrics. Both models converge within their time budgets, with T5-base achieving higher F1 (85.96% vs. 79.60%).

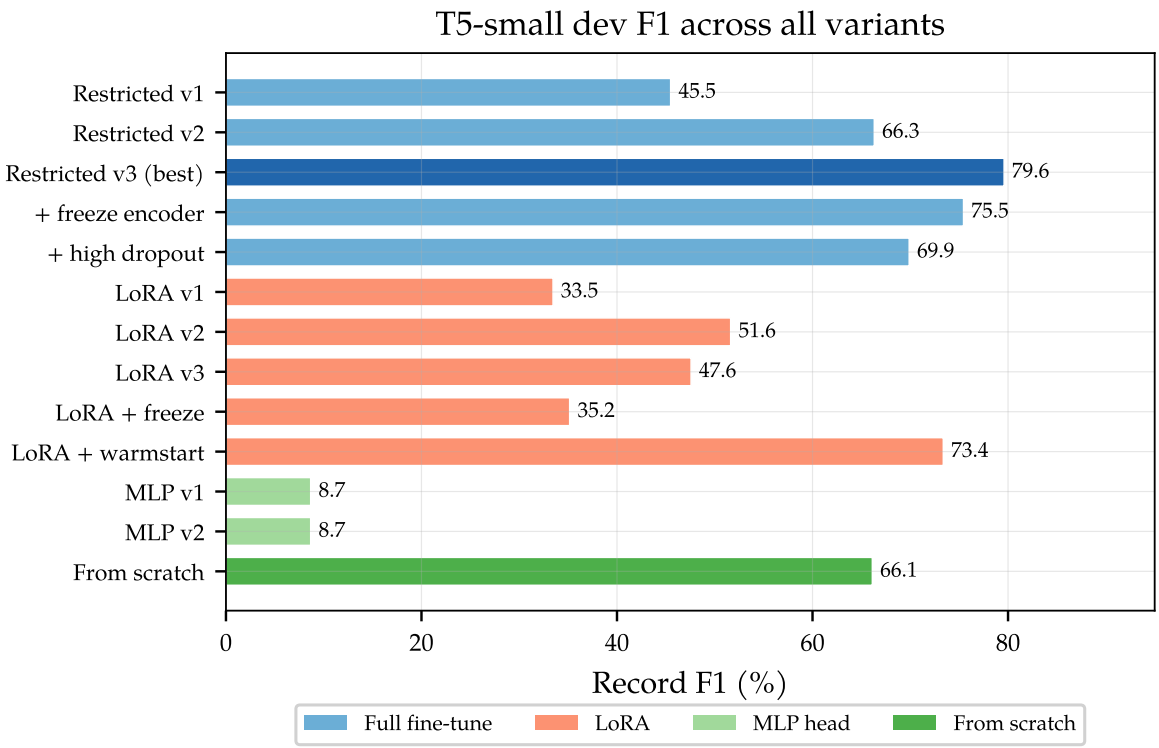

Figure 2: Record F1 on the dev set across all T5-small variants. Full fine-tuning with schema augmentation (Restricted v3) achieves the highest F1 (79.6%). LoRA with warm-start initialization (73.4%) approaches full fine-tuning performance. MLP projection heads underperform at 8.7%.

System	Method	Params (M)	LR	Batch	Schema	Dropout	WD	Beams	Epochs	Time	Query EM (%)	F1 (%)
Dev Results — T5-small Fine-tuned												
Full fine-tune (restricted vocab)												
Restricted v1	Full	60.5	3e-5	32	–	0.1	0.01	4	29	45m	–	45.49
Restricted v2	Full	60.5	1e-4	64	–	0.1	0.01	4	59	55m	–	66.32
Restricted v3 (best)	Full	60.5	3e-4	64	✓	0.1	0.01	4	79	75m	**58.80**	**79.60**
Restricted v5 + freeze enc	Full	41.6	3e-4	64	–	0.1	0.01	4	79	75m	–	75.45
Restricted v7	Full	60.5	1e-4	64	✓	0.2	0.01	4	79	75m	–	69.89
LoRA adapters												
LoRA v1 (r=16, q/v)	LoRA	0.6	3e-4	128	–	0.1	0.01	4	79	72m	–	33.46
LoRA v2 (r=16, q/v)	LoRA	0.6	3e-4	64	✓	0.1	0.01	4	79	80m	–	51.63
LoRA v3 (r=32, q/k/v/o)	LoRA	2.4	2e-4	64	–	0.1	0.01	4	79	93m	–	47.58
LoRA + freeze encoder	LoRA	0.6	5e-4	128	–	0.1	0.01	4	79	86m	–	35.16
LoRA + warmstart	LoRA	0.6	3e-4	128	–	0.1	0.01	4	79	90m	–	73.38
MLP projection head												
MLP v1 (dim=1024)	MLP	61.6	1e-4	64	–	0.1	0.01	4	31	28m	–	8.67
MLP v2 (dim=1024)	MLP	61.6	1e-4	64	✓	0.1	0.01	4	31	27m	–	8.67
Dev Results — T5-small From Scratch												
From-scratch	Full	60.5	3e-4	32	–	0.1	0.01	4	98	8h	50.43	66.12
Test Results												
T5 fine-tuning	Full	60.5	3e-4	64	✓	0.1	0.01	4	79	–	0.00	55.25
T5 from scratch	Full	60.5	3e-4	32	–	0.1	0.01	4	98	–	0.00	59.16

Table 11: T5-small development and test results (Parts 1 & 2, trained on VM 1). Params (M) = tunable parameters in millions. WD = weight decay, Beams = num_beams during decoding. Best dev result in bold.

System	Method	Params (M)	LR	Schema	Dropout	Label Sm.	WD	Beams	Epochs	Time	Query EM (%)	F1 (%)
Dev Results — T5-base Fine-tuned												
Full fine-tune												
Base sweep best	Full	222.9	6.19e-5	✓	0.05	0.15	0.009	1	79	75m	**73.39**	**85.96**
Base sweep v2	Full	222.9	5.52e-5	✓	0	0.15	0.002	1	79	75m	–	84.31
Base sweep v3	Full	222.9	2.59e-4	–	0.2	0.1	0.022	1	71	58m	73.33	83.05
Base sweep v4	Full	222.9	5.50e-5	–	0	0.15	0.001	1	31	26m	–	75.77
MLP projection head												
Base MLP-512 v1	MLP	223.7	2.35e-4	–	0.15	0.15	0.004	1	123	93m	71.33	80.76
Base MLP-512 v2	MLP	223.7	7.83e-4	–	0.15	0.05	0.002	2	43	32m	69.33	79.56
Base MLP-1024	MLP	224.5	1.28e-4	–	0.1	0.05	0.019	4	147	109m	69.33	78.86
LoRA adapters												
Base LoRA r=32 (q/k/v/o)	LoRA	7.1	1.29e-5	✓	0.05	0.05	0.068	2	335	283m	61.33	76.48
Base LoRA r=32 (q/k/v/o)	LoRA	7.1	1.22e-4	✓	0.05	0.15	0.027	3	31	27m	–	66.69
Base LoRA r=16 (q/v)	LoRA	1.8	3.07e-5	–	0.05	0	0.021	4	71	57m	32.00	55.51

Table 12: T5-base fine-tuning development results from W&B random sweep (222M total params, batch = 32, trained on VM 2). WD = weight decay, Beams = num_beams during decoding. Params (M) = tunable parameters in millions. Best result in bold.

DPO post-training results. Table 13 reports DPO results (Section 2.2) alongside the SFT baseline. DPO did not improve over the SFT checkpoint.

System	SQL EM (%)	Record EM (%)	Record F1 (%)
Dev Results			
SFT baseline (T5-base)	73.39	84.76	85.96
Full fine-tune (β=0.3, LR=3.82e-6, WD=8.95e-4, batch=16)			
Epoch 0 (= SFT)	68.45	80.69	82.18
Best (epoch 2)	71.24	83.69	**84.82**
Final eval (best ckpt, beam=4)	68.67	81.12	82.42
LoRA r=16, q/k/v/o (β=0.3, LR=1.16e-7, WD=0.071, batch=32)			
Epoch 0 (= SFT)	73.39	84.76	85.96
Best (epoch 0–5, no change)	73.39	84.76	85.96
Final eval (best ckpt, beam=4)	61.59	74.89	75.75
Test Results			
DPO (best checkpoint)	–	–	–

Table 13: DPO results on T5-base (Section 2.2). The SFT baseline is "Base sweep best" from Table 12. **Full fine-tune:** training peaked at epoch 2 (F1 = 84.82%) before degrading; early stopping triggered at epoch 7. The final beam-search evaluation of the best checkpoint (82.42%) fell further below the SFT baseline (85.96%). **LoRA:** the low learning rate (1.16e-7) caused effectively no policy update over 6 epochs—dev metrics remained identical to the SFT baseline. However, the final beam-search evaluation dropped to 75.75%, a 10.2 pp gap attributed to LoRA merge-and-save corruption and the greedy-vs-beam decoding mismatch (see Table 18).

Four factors likely explain why DPO underperformed:

1. **Narrow output distribution.** The restricted 598-token SQL vocabulary already constrains the model to a small output space. DPO's advantage of steering away from bad outputs is diminished when the model has few degrees of freedom.
2. **Preference data quality.** Rejected candidates were either model-generated errors (Phase A) or rule-based perturbations (Phase B). Neither captures the subtle SQL structural errors (wrong JOIN patterns, missing subqueries) that dominate the remaining 14% SFT error rate (Table 15).
3. **Reward hacking / overfitting (full fine-tune).** The reward margin grew from 1.6 to 12.2 over 8 epochs while dev F1 peaked at epoch 2 and collapsed to 71.7% by epoch 7. The policy learned to maximize the DPO reward signal without improving SQL correctness, a form of reward hacking on the fixed preference dataset.
4. **Insufficient LoRA learning rate.** The LoRA trial used LR = 1.16e-7, which was too low to move the adapter weights meaningfully—reward accuracy reached only 54% (barely above random) and dev metrics remained identical to the SFT baseline across all 6 epochs. The 10.2 pp F1 drop at final evaluation (85.96%→75.75%) is attributed to LoRA merge-and-save corruption and the training-time greedy vs. final beam-search decoding mismatch.

ICL sensitivity to k: For ICL, please provide a plot of the Record F1 on the development set that the model achieved with different values of k. The x-axis should be k, and the y-axis the Record F1. The prompts and examples used for this plot should correspond to the ones you described in Subsection 3.1.

System	Query EM (%)	F1 (%)
Dev Results		
k-shot prompting (random selection)		
Random $k = 0$ (zero-shot)	0.00	12.60
Random $k = 1$	0.00	12.60
Random $k = 3$	0.00	11.96
BM25 per-query selection		
BM25 $k = 3$ **(best)**	0.00	**17.35**
Ablation study (BM25 $k = 3$)		
No instructions	0.00	11.80
No schema	0.00	11.80
Schema only (no instr., no examples)	0.00	11.37
Test Results		
BM25 $k = 3$	0.00	13.93

Table 14: LLM prompting development and test results (Part 3). Query EM and Record F1 on the development set for Gemma-2B with various k-shot and example selection strategies. Best dev result in bold. Ablations remove individual prompt components from the best (BM25 $k = 3$) configuration.

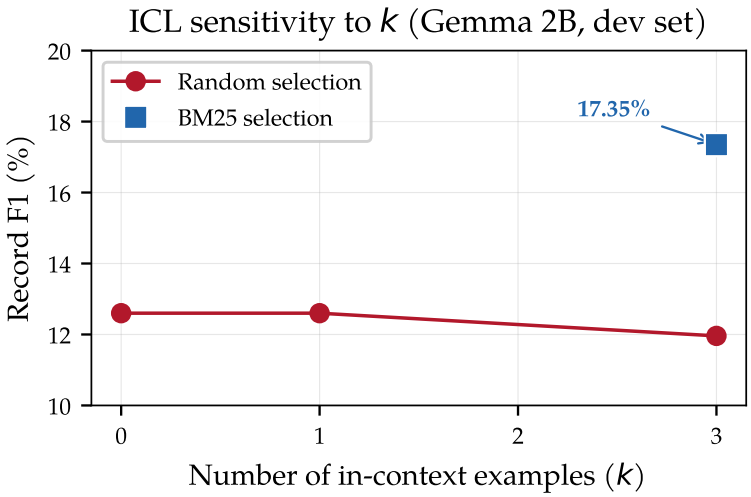

Figure 3: ICL sensitivity to k on the dev set. Random example selection (red) shows flat or slightly declining F1 as k increases. BM25 per-query selection at $k = 3$ (blue) achieves 17.35%, a 45% relative improvement over random $k = 3$, demonstrating that retrieval-based example selection is critical for this task.

Qualitative Error Analysis: We identified five common error patterns across the three models, summarized in Table 15.

Error Type	Models		Example Of Error	Error Description	Statistics

Missing comparison operator	T5-FT, T5-Scr	NL
		GT
		Pred
			"flights arriving before 6pm"
			`arrival_time < 1800`
			`arrival_time  1800`
				Drops <, >, <=, >= from predicates, producing invalid SQL. SentencePiece treats angle brackets as special tokens.	T5-FT: 26/466 (5.6%)
					T5-Scr: 27/466 (5.8%)

Query truncation	ICL	NL
		GT
		Pred
			"afternoon flights from washington to boston"
			`SELECT DISTINCT flight_1.flight_id FROM flight flight_1 ...`
			`SELECT DISTINCT flight_1.flight_id`
				Only a `SELECT` fragment generated, missing `FROM`/`WHERE`. Caused by Gemma 2B `max_new_tokens` limit.	ICL: 383/466 (82.2%)

Wrong table/column reference	T5-FT, ICL	NL GT Pred	"flight from boston to denver" `...city city_1, ...city city_2` `...city city_2` (missing `city_1`)	References aliases or columns not in `FROM`. Model generates plausible patterns but omits table declarations.	T5-FT: 16/466 (3.4%) ICL: 5/466 (1.1%)
Wrong JOIN structure	T5-FT, T5-Scr, ICL	NL GT Pred	"flights from boston to SF with max stops" `stops = (SELECT MAX(stops)...)` `stops = 0` (returns 4 instead of 2)	Executes but returns wrong records: incorrect subqueries, missing joins, or wrong aggregation on multi-table queries.	T5-FT: 83/466 (17.8%) T5-Scr: 144/466 (30.9%) ICL: 35/466 (7.5%)

Wrong predicate values	T5-FT, T5-Scr, ICL	NL
		GT
		Pred
			"flights available tomorrow from denver to philadelphia"
			`month_number = 1 AND day_number = 20`
			`month_number = 2 AND day_number = 20`
				Correct SQL structure but wrong literals in `WHERE` (month, city, time). Subset of semantic errors, identified by matching `FROM`.	T5-FT: 83/466 semantic
					T5-Scr: 144/466 semantic
					ICL: 35/466 semantic

Table 15: Qualitative error analysis on the dev set (466 queries) for all three models. Statistics use COUNT/TOTAL format where TOTAL is the full dev set size. Categories are mutually exclusive: each query is classified into exactly one category per model.

5 Engineering Issues

Table 16 documents the performance-related engineering issues encountered during development, along with their root causes and resolutions.

Issue	Description	Resolution
GPU OOM from concurrent sessions	AutoBatch maximizes VRAM for the first GPU task; a second task (training, eval, or inference) crashes with CUDA OOM because no mechanism prevents concurrent GPU access.	File-based GPU lock using Linux `flock`. All training entry points acquire the lock automatically; a second task blocks and waits instead of crashing.
Training process hung after completion	Training PID blocks indefinitely on `futex_wait_queue_me` during `gc.collect()` / `torch.cuda.empty_cache()` after W&B marks the run FINISHED, caused by a PyTorch/CUDA threading deadlock during GPU memory cleanup. Sequential scripts cannot proceed.	Watchdog co-process polls W&B status every 60 s and kills the training process if still alive 5 min after FINISHED. Exit codes 137/143 treated as success.
Buffered stdout under nohup	Python buffers stdout when not connected to a TTY (piped or redirected by `nohup`), so log files show no output for long periods, making monitoring ineffective.	Set `PYTHONUNBUFFERED=1` before launching Python in the training script.

Table 16: Performance issues encountered during development.

Issue	Description	Resolution
BOS token mismatch	T5 decoder received pad token (ID 0) as start-of-sequence during inference instead of `<extra_id_0>` (ID 32099) used during training. The model never learned to predict `SELECT` after pad, so every prediction started mid-query, causing 100% SQL execution error rate.	Added `decoder_start_token_id=32099` to all `model.generate()` calls so inference uses the same BOS token as training.
`no_repeat_ngram_size` blocks valid SQL	`no_repeat_ngram_size=3` structurally prevented generating valid multi-join SQL. 100% of ground truth queries contain repeated 3-grams (e.g., `airport_code AND` in multiple join conditions), so the constraint corrupted every multi-join query.	Disabled `no_repeat_ngram_size` (set to 0).
LoRA checkpoint drops adapter weights	Post-training final eval deleted the in-memory model, reloaded from checkpoint, then re-applied LoRA with fresh random adapters. Checkpoints intentionally stripped `lora_A/lora_B` keys, so reloaded models had untrained adapters, collapsing F1 from ∼0.516 to 0.118.	Removed LoRA key stripping from `state_dict()`; checkpoints now preserve adapter weights. Updated reload path to re-apply LoRA then load checkpoint including adapters.
MLP head device mismatch	`T5ForFlightSQLWithMLP` creates MLP and LayerNorm layers on CPU while the base T5 model is already on GPU, causing `RuntimeError` at the first forward pass when CUDA tensors mix with CPU parameters.	Added `model.to(device)` after creating the MLP wrapper in both initial creation and checkpoint loading paths.
Repetition penalty too aggressive	`repetition_penalty=2.5` divided each token's logit by 2.5 after first use. SQL reuses tokens like `AND`, `.`, `=` extensively, so heavy penalization distorted the probability distribution away from valid SQL patterns.	Set `repetition_penalty=1.0` (disabled).
Repetition penalty zero config	Config set `repetition_penalty=0` instead of 1.0. Since HuggingFace applies the penalty as a divisor to token logits, this would cause division by zero at runtime.	Changed to `repetition_penalty=1.0` (no penalty) before launching any runs.
MLP checkpoint state_dict mismatch	`save_model()` saved the full MLP wrapper's `state_dict()` (including `mlp_norm`, `mlp.*` keys) instead of the inner T5 model's. Subsequent non-MLP trials crashed loading checkpoints with unexpected keys, wasting 4 of 8 sweep trials.	Changed `save_model()` to always save `inner.state_dict()` (vanilla T5 weights), consistent with the LoRA path.
`max_new_tokens` truncation	`max_new_tokens=256` truncated 26% of dev targets (120/466 queries exceed 256 tokens; max target length is 511). Complex multi-join queries were cut off mid-generation, producing incomplete SQL.	Increased `max_new_tokens` to 512 to cover the full target length range.
SentencePiece comma spacing artifact	`batch_decode` collapses `" , "` (space-comma-space) to `", "` (comma-space) because the SentencePiece decoder consumes the space sentinel token before commas. Since 100% of ground truth queries use `" , "`, raw Query EM dropped to ∼3% despite the model producing correct token sequences internally.	Regex post-processing after `batch_decode`: `re.sub(r'(?<=[a-zA-Z0-9_()])\s*,\s+', ' , ', sql)`. Corrected Query EM: 58.80% (fine-tuned), 50.43% (from scratch).

Table 17: Model issues encountered during development.

Issue	Description	Resolution
OOV CUDA assert in DPO loss	`remap_targets` returned -1 for tokens outside the restricted vocab (0.2% of triplets), causing `torch.gather` out-of-bounds on GPU.	Clamp unmapped indices to 0 and mask them out of the log-probability sum.
Auto batch probe OOM	Probe measured `memory_allocated` (post-cleanup) instead of peak, and tested short sequences; training OOM'd on longer batches.	Use `max_memory_allocated` with worst-case (longest) sequences.
Slow preference data gen	46K SQL evaluations took $\sim$90 min due to SQLite file locking + per-query connection overhead.	In-memory SQLite copy (`sqlite3.backup`) + gold record disk cache. Reduced to $\sim$20 min.

Table 18: DPO-specific engineering issues (Section 2.2).